
`github.com/mg-i-roc-maz/corsi · microservizi/lezione-3`

LEZIONE 3 DI 8 · 3 ORE

Progettare e far comunicare i servizi

Ogni servizio con i suoi dati, contratti chiari tra i servizi e un unico ingresso per chi sta fuori.

Le tre ore di oggi

Teoria · 1h15

Teoria

Database per servizio, contratti e versioni delle API, comunicazione sincrona, service discovery, API Gateway.

Laboratorio · 1h35

Laboratorio

orders chiama catalog via HTTP, Nginx come gateway, discovery con il DNS di Compose, guasti.

Ultimi 10'

Una domanda per chiudere: «cosa si rompe se...?»

DA DOVE RIPARTIAMO

Due servizi in due container. Ma orders accetta ordini senza sapere se i prodotti esistono.

CUCITURA 1

Nome e prezzo: oggi orders li chiede a catalog.

CUCITURA 2

Le scorte: oggi catalog le scala su richiesta di orders.

CUCITURA 3

L'email di conferma: lezione 7, con un evento.

La tentazione: un database per tutti

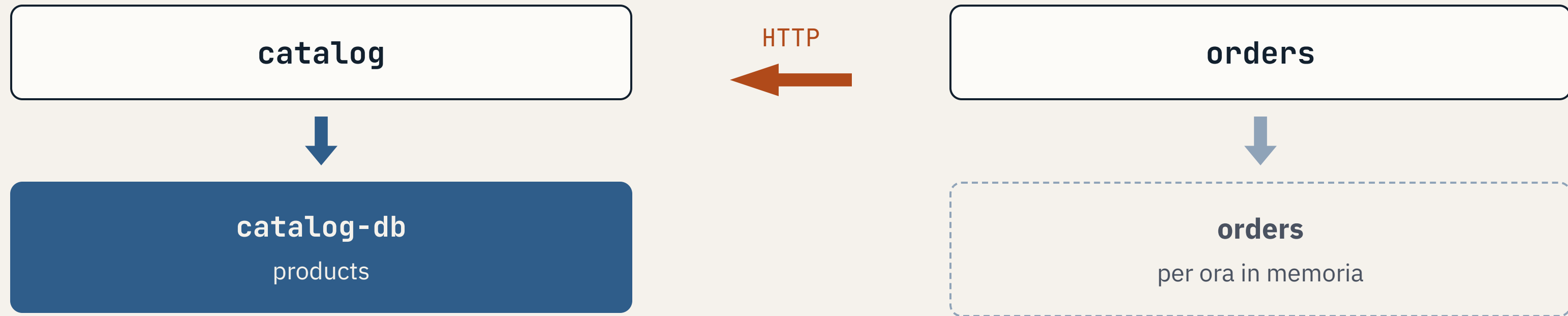


Sembra comodo: orders legge **products** con una JOIN. Ma:

- catalog rinomina una colonna e **orders si rompe**
- i due servizi vanno rilasciati **insieme**
- non si capisce più **di chi** sono i dati
- un servizio lento rallenta il database di tutti

È un monolite travestito: il database è diventato il punto di accoppiamento.

Ogni servizio possiede i suoi dati



Solo catalog legge e scrive **products**. Può cambiare tabelle e database quando vuole.

Chi ha bisogno dei prodotti passa dalla sua **API**: è l'unico contratto pubblico.

E se mi servono i dati di un altro?

Strategia	Come	Quando
Chiedere	Una chiamata all'API del proprietario, quando serve	Dati che cambiano spesso e vanno freschi: le scorte. Oggi
Fotografare	Copiare il valore nel proprio dato, al momento giusto	Il prezzo pagato va nell'ordine e non cambia più. Oggi
Ricevere	Una copia locale aggiornata dagli eventi del proprietario	Letture frequenti, tollerano un piccolo ritardo. Lezione 7



Una riga d'ordine contiene **productName** e **unitPriceCents**: se domani catalog cambia il prezzo, l'ordine di ieri resta com'era. Non è una copia sbagliata: è un fatto storico.

Niente transazione unica tra due servizi

Nel monolite

```
BEGIN;  
  UPDATE products SET stock = ...;  
  INSERT INTO orders ...;  
COMMIT; # tutto o niente
```

Con due servizi

orders chiede a catalog di prenotare **p-1**: ok. Poi **p-3**: scorte finite.

p-1 è già stato tolto dal magazzino, e nessun ROLLBACK lo rimette a posto.

Bisogna **compensare**: un'operazione che annulla quella già fatta. È l'idea della **Saga**, che vediamo nella lezione 8.

L'ultimo monitor, finalmente

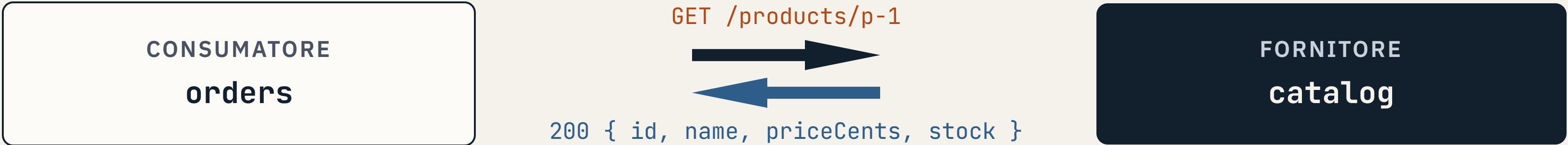
Lezione 1: due clienti ordinano l'ultimo monitor nello stesso istante. Con due servizi, «leggo le scorte, poi le scalo» non funziona più: tra le due operazioni passa un'altra richiesta.

```
-- services/catalog/src/store/postgres.ts
UPDATE products
  SET stock = stock - $2
  WHERE id = $1 AND stock ≥ $2
RETURNING ...
```

Una sola istruzione: PostgreSQL controlla e sottrae insieme. Se le scorte non bastano, la WHERE non trova righe e non cambia nulla.

Il controllo sta **dove stanno i dati**: in catalog, non in orders.

Un'API è un contratto



Il contratto dice	Esempio
Percorsi e metodi	GET /products/:id, POST /products/:id/reservations
Forma dei dati	priceCents è un intero, in centesimi
Codici di risposta	404 se non esiste, 409 se le scorte non bastano
Formato degli errori	{ "error": ".." } in tutti i servizi

Modifiche compatibili e no

Una modifica è **compatibile** se i consumatori esistenti continuano a funzionare senza cambiare una riga.

Compatibili

- aggiungere un campo alla risposta
- aggiungere una rotta
- aggiungere un campo **facoltativo** alla richiesta
- accettare più valori di prima

Incompatibili

- togliere o rinominare un campo
- cambiarne il tipo o l'unità
- rendere obbligatorio un campo
- cambiare il significato di un codice

Severo quando mandi, tollerante quando ricevi

```
// services/orders/src/catalog-client.ts
export interface CatalogProduct {
  id: string;
  name: string;
  priceCents: number;
  stock: number;
  // description? Non ci serve: la ignoriamo
}
```

orders legge **solo i campi che usa**. Se catalog aggiunge **category** o **imageUrl**, orders non se ne accorge nemmeno.

È il **Tolerant Reader**: un consumatore che ignora quello che non conosce sopravvive ai cambiamenti del fornitore.

Versionare un'API

Dove sta la versione	Esempio	Pro e contro
Nel percorso	<code>/api/v1/products</code>	Visibile, facile da instradare. Il più usato
Nell'header	<code>Accept: application/vnd.shop.v2+json</code>	URL puliti, ma meno visibile
Nella query	<code>/products?version=2</code>	Semplice, ma si dimentica facilmente

Regole pratiche

- nuova versione **solo** per modifiche incompatibili
- la vecchia resta finché ci sono consumatori
- si annuncia la data di dismissione

Nel nostro negozio

Il gateway espone **/api/v1/...** I servizi dentro non sanno nulla della versione.

Quale codice risponde orders?

Situazione	Codice	Perché
Corpo non valido: email sbagliata, quantità 0	400	Errore di forma, lo vede lo schema
Prodotto inesistente	422	Forma giusta, contenuto impossibile
Scorte insufficienti	409	Conflitto con lo stato attuale
catalog non risponde	503	orders c'è, ma non può lavorare
Il gateway non raggiunge orders	502	Lo decide il gateway, non orders
Tutto ok	201	Ordine creato, con Location

Rotte pubbliche e rotte interne

Rotta di catalog	Chi la usa
GET /products	Clienti e orders
GET /products/:id	Clienti e orders
POST, PUT, DELETE /products	Amministrazione
POST /products/:id/reservations	Solo orders

Prenotare scorte non è un'operazione per i clienti: chiunque potrebbe svuotare il magazzino.

Il gateway la **blocca** con un 403. Dentro la rete di Compose, orders la raggiunge comunque.



Il nome: **reservations** è una sottorisorsa del prodotto, e POST ne crea una. Meglio di un verbo nel percorso, come /reserveProduct.

COMUNICAZIONE SINCRONA

Chiedo e aspetto la risposta.

Semplice da scrivere e da capire: è una chiamata di funzione che passa dalla rete. Ma mentre catalog lavora, orders **aspetta**, e anche il cliente.

La latenza si somma

Un ordine con **3 righe**: orders chiama catalog una volta per leggere e una per prenotare, per ogni riga.

GET p-1

GET p-2

GET p-3

POST p-1

POST p-2

POST p-3

6 chiamate in fila

Se ognuna costa 5 ms, l'ordine aspetta almeno **30 ms** solo di rete. Con 20 righe, 200 ms.

Come si riduce

Una chiamata per tanti prodotti (**?ids=p-1,p-2**), oppure chiamate in parallelo con **Promise.all**.

La disponibilità si moltiplica

Se orders, per rispondere, ha bisogno di catalog, è disponibile solo quando lo sono **tutti e due**.

Catena	Disponibilità	Fermo in un anno
catalog da solo, 99,9%	99,9%	circa 9 ore
orders → catalog, 99,9% ciascuno	99,8%	circa 17 ore e mezza
5 servizi in catena, 99,9% ciascuno	99,5%	circa 44 ore

Più servizi chiamano altri servizi in modo sincrónico, più facile è che qualcosa sia giù.

Quando catalog non risponde bene

1

Connessione rifiutata

catalog è spento. fetch fallisce subito:
ECONNREFUSED.

orders → 503 in pochi millisecondi.

2

Risposta d'errore

catalog è acceso ma ha un problema:
risponde **500**.

orders → 503, e lo scrive nel log.

3

Nessuna risposta

catalog è bloccato. La connessione
resta aperta e fetch **aspetta**.

orders si blocca con lui. Il caso peggiore.

Oggi gestiamo i casi 1 e 2. Il caso 3 richiede un **timeout**: è il punto di partenza della lezione 4.

Sincrona o asincrona?

	Sincrona: HTTP	Asincrona: messaggi
Come	Chiedo e aspetto	Pubblico un messaggio e vado avanti
Serve che l'altro sia vivo	Sì, adesso	No: lo leggerà quando torna
Risposta	Subito, nella stessa richiesta	Dopo, o mai
Va bene per	«Quanto costa? Ci sono scorte?»	«Manda l'email di conferma»
Nel corso	Lezioni 3 e 4	Lezioni 7 e 8, con RabbitMQ

Come fa orders a trovare catalog?

Un IP scritto nel codice

`http://172.18.0.4:3001`

Cambia a ogni avvio dei container. Con due copie di catalog, quale?

Un nome, risolto dal DNS

`http://catalog:3001`

Compose ha un DNS interno: il nome del servizio diventa l'indirizzo dei suoi container.

```
# compose.yaml
orders:
  environment:
    CATALOG_URL: http://catalog:3001
```

L'indirizzo è **configurazione**: arriva da una variabile d'ambiente. Con `npm run dev` vale `localhost:3001`, in Compose `catalog:3001`.

Dentro il DNS di Compose



Cosa succede

Con due copie, il nome restituisce due indirizzi

Un container ricreato prende un IP nuovo

Nginx risolve i nomi **all'avvio**

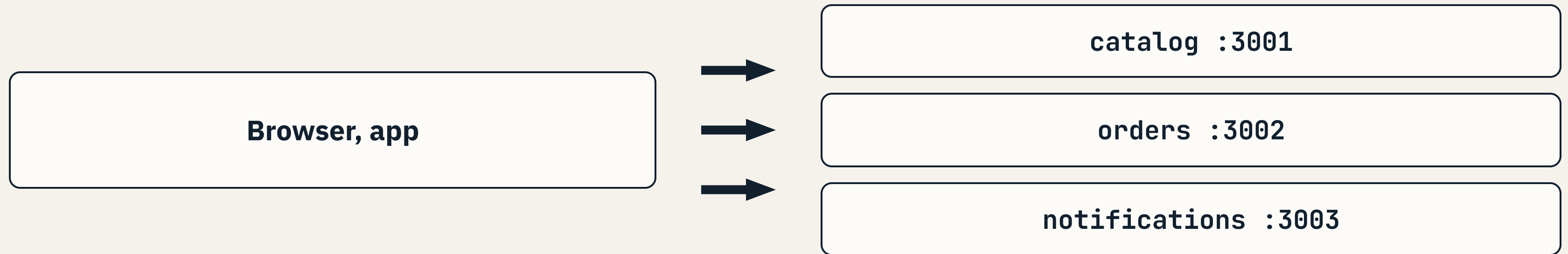
Conseguenza

Le richieste si dividono tra le copie

Il nome resta lo stesso: niente da cambiare

Dopo uno scale o un riavvio: restart del gateway

Senza un ingresso unico



- il client deve conoscere **tutti** gli indirizzi, e cambiare quando cambiano
- sicurezza, log e limiti vanno ripetuti in **ogni** servizio
- anche le rotte interne, come le prenotazioni, sono raggiungibili da fuori

L'API Gateway



Un solo indirizzo pubblico. Dietro, il gateway decide a quale servizio inoltrare ogni richiesta: è un **reverse proxy** con qualche compito in più.

Cosa fa un gateway, e cosa no

Sì: compiti trasversali

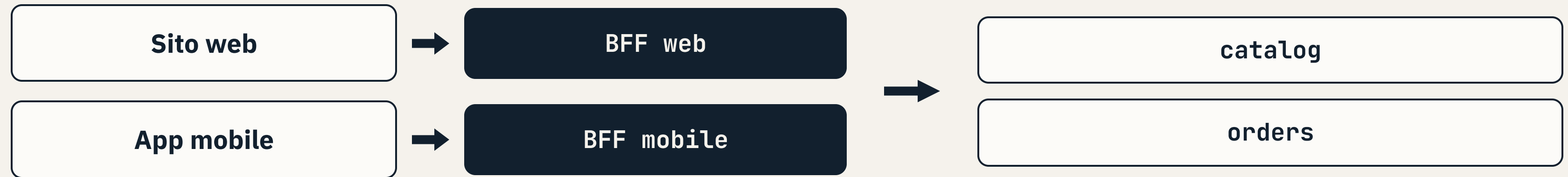
Compito	Oggi
Instradamento	/api/v1/products → catalog
Nascondere l'interno	403 sulle prenotazioni
Riscrivere	Location verso l'indirizzo pubblico
Log centralizzati	Ogni richiesta, con il servizio
Limiti di traffico	Esercizio extra 3
TLS, autenticazione	Fuori corso

No: logica di business

Calcolare un totale, decidere se un ordine è valido, unire dati di più servizi in modo complesso.

Se il gateway cresce, diventa un nuovo monolite in mezzo a tutto.

Un gateway per ogni client: BFF



Backend for Frontend: ogni tipo di client ha il suo punto d'ingresso, che prepara i dati come servono a lui. Il mobile vuole risposte piccole, il sito più dettagli.

Nginx in poche righe

```
events {}
http {
    server {
        listen 80;

        location /api/v1/products {
            proxy_pass http://catalog:3001/products;
        }

        location /api/v1/orders {
            proxy_pass http://orders:3002/orders;
        }
    }
}
```

Blocco	Significato
http	Configurazione del server web
server	Un sito, in ascolto su una porta
location	Quali percorsi gestire
proxy_pass	A chi inoltrarli

Ogni direttiva finisce con il **punto e virgola**: dimenticarlo è l'errore più comune.

Come nginx sceglie la location

Forma	Significato	Priorità
<code>location = /health</code>	Solo esattamente /health	1, vince subito
<code>location ~ ^/api/.../reservations</code>	Espressione regolare	2, la prima che combacia
<code>location /api/v1/products</code>	Prefisso: tutto ciò che inizia così	3, il più lungo

proxy_pass con un percorso

```
location /api/v1/products
proxy_pass http://catalog:3001/products;
```

Il prefisso viene **sostituito**: /api/v1/products/p-1 → /products/p-1.

Senza percorso

```
proxy_pass http://catalog:3001;
```

Il percorso passa **com'è**: catalog riceverebbe /api/v1/products e risponderebbe 404.

Reverse proxy, load balancer, gateway

	Cosa fa	Esempio oggi
Reverse proxy	Riceve le richieste al posto dei server e le inoltra	nginx davanti ai servizi
Load balancer	Divide le richieste tra più copie dello stesso servizio	due copie di catalog
API Gateway	Reverse proxy per API: versioni, regole, sicurezza, limiti	/api/v1, 403, 429

Spesso sono lo **stesso programma** con configurazioni diverse. Nginx oggi fa tutte e tre le cose.

Verifichiamo: dati e contratti

1 Perché orders non deve leggere la tabella products?

2 Perché l'ordine salva il prezzo invece di rileggerlo?

3 Rinominare priceCents in price è compatibile?

4 Perché reserve è una sola istruzione SQL?

Verifichiamo: comunicazione e gateway

1 Perché due servizi al 99,9% in catena danno meno del 99,9%?

2 Da orders, perché `http://catalog:3001` e non `localhost`?

3 Quale caso di guasto è il peggiore, e perché?

4 Cosa NON deve fare un API Gateway?

Glossario

Termine	Significato
Database per servizio	Ogni servizio possiede i suoi dati; gli altri passano dalla sua API.
Contratto	Percorsi, campi, tipi e codici su cui consumatore e fornitore si accordano.
Compatibilità	Una modifica che non obbliga i consumatori a cambiare.
Tolerant Reader	Consumatore che ignora i campi che non conosce.
Compensazione	Operazione che annulla un passo già fatto in un altro servizio.
Service discovery	Trovare dove gira un servizio, per esempio con il DNS.
Reverse proxy	Server che riceve le richieste al posto di altri e le inoltra.
API Gateway	Unico ingresso alle API, con instradamento e regole comuni.

Laboratorio

10'

1 · Il contratto

Leggere la rotta di prenotazione e il client di orders.

40'

2 · orders chiama catalog

Tre TODO: getProduct, reserve, POST /orders.

25'

3 · Il gateway

nginx.conf e compose.yaml: un solo ingresso.

15'

4 · Discovery e guasti

Due copie di catalog, poi lo spegniamo.

```
cd microservizi/lezione-3/start/services/orders → npm install → npm test
```

Il repo della lezione

github.com/mg-i-roc-maz/corsi

```
cd corsi
git pull
cd microservizi/lezione-2/start
docker compose down
cd ../../lezione-3/start
code .
```

microservizi/lezione-3/

Cartella	Contenuto
slides.pdf	Queste slide, in PDF
start/	Si lavora qui
solution/	La soluzione, per riallinearsi
extra/	Esercizi extra, test e soluzioni

Il progetto Compose si chiama **shop** in tutte e due le lezioni: si ferma prima quello della lezione 2.

Cosa c'è in start

gateway/nginx.conf

services/catalog/

src/routes.ts

src/store/postgres.ts

services/orders/

src/catalog-client.ts

src/routes.ts

test/fake-catalog.ts

compose.yaml

http/shop.http

TODO · Parte 3

nuova: reservations

reserve atomica

TODO · Parte 2

TODO · Parte 2

catalog finto

TODO · Parte 3

tutto dal gateway

È la soluzione della lezione 2, più:

- la prenotazione di scorte in catalog, già pronta
- il client HTTP di orders, da completare
- il gateway, da configurare

1.1 · Leggi la nuova rotta di catalog

```
POST /products/p-3/reservations
{ "quantity": 2 }
```

```
200 { "id": "p-3", ..., "stock": 3 }
```

```
404 { "error": "Prodotto non trovato" }
```

```
409 { "error": "Scorte insufficienti",
      "available": 1 }
```

Apri e leggi:

- services/catalog/src/routes.ts, in fondo
- reserve in src/store/postgres.ts



Perché la prenotazione è una **sola** UPDATE e non una SELECT seguita da una UPDATE?

1.2 · Il contratto visto da orders

```
// services/orders/src/catalog-client.ts
export interface CatalogClient {
  getProduct(id: string):
    Promise<CatalogProduct | undefined>;
  reserve(id: string, quantity: number):
    Promise<ReservationResult>;
}

export class CatalogUnavailableError
  extends Error {}
```

orders non parla HTTP nelle rotte: chiama un **CatalogClient**. Tutti i dettagli di rete stanno in questo file.

Le risposte di catalog diventano valori semplici: un prodotto, **undefined**, un esito. I problemi di rete diventano **CatalogUnavailableError**.

2.1 · Parti dai test

```
cd services/orders  
npm install  
npm test
```

Puoi lavorare nel Dev Container o con Node sul computer: non serve Docker.

ALL'INIZIO

pass 3 · # fail 10

OBIETTIVO

pass 13 · # fail 0

2.2 · fetch in trenta secondi

```
const res = await fetch(url, {  
  method: "POST",  
  headers: { "content-type": "application/json" },  
  body: JSON.stringify({ quantity: 2 }),  
});  
res.status // 200, 404, 409...  
res.ok // true se 200-299  
await res.json() // il corpo, letto UNA volta
```

Due modi di fallire

Errore HTTP (404, 500): fetch **non** lancia eccezioni.
Devi guardare res.status.

Errore di rete (catalog spento): fetch **lancia**. La
funzione call() lo trasforma già in
CatalogUnavailableError.

2.3 · TODO 1 e 2: il client

Risposta di catalog	getProduct restituisce
200	il corpo JSON
404	undefined
altro	lancia CatalogUnavailableError

Risposta di catalog	reserve restituisce
200	{ status: "reserved" }
404	{ status: "not_found" }
409	{ status: "insufficient_stock", available }
altro	lancia CatalogUnavailableError



Controlla 404 e 409 **prima** di res.ok: anche loro hanno res.ok uguale a false.

Dopo i TODO 1 e 2 devono passare i quattro test di **catalog-client.test.ts**.

2.4 · TODO 3: POST /orders

1. Per ogni item: **getProduct**. Manca? **422**. C'è? Una riga con nome e prezzo.
2. Per ogni riga: **reserve**. Non riuscita? **409**.
3. Salva l'ordine: già scritto sotto il TODO.

Perché prima tutte le letture e poi tutte le prenotazioni, e non riga per riga?

Già pronto

Il **try/catch** intorno trasforma `CatalogUnavailableError` in **503**. Tu pensi solo al caso in cui catalog risponde.

```
npm test
```

2.5 · Come fanno i test senza catalog?

```
// test/orders.test.ts
const catalog = fakeCatalog();
const app = buildApp({ catalog });

// catalog spento, per finta
buildApp({ catalog: fakeCatalog({ down: true }) })
```

```
// src/server.ts, quello vero
buildApp({ catalog:
  createHttpCatalogClient(catalogUrl) })
```

buildApp **riceve** il client da fuori: i test passano quello finto, il server quello HTTP.

Così si prova anche il caso **catalog spento**, che con il servizio vero sarebbe scomodo da riprodurre.

3.1 · TODO 1–3: gateway/nginx.conf

Pubblico	Inoltrato a
/api/v1/products...	catalog:3001/products...
/api/v1/orders...	orders:3002/orders...
.../reservations	403, il gateway risponde da solo
/health	200, già scritto
tutto il resto	404, già scritto

```
location /api/v1/products {
    proxy_pass
        http://catalog:3001/products;
    proxy_redirect
        /products/ /api/v1/products/;
}
```

Il resto lo trovi nei commenti del file.

3.2 · TODO in compose.yaml

```
gateway:
  image: nginx:1.28
  volumes:
    - ./gateway/nginx.conf:/etc/nginx/nginx.conf:ro
  ports:
    - "8080:80"
  depends_on: [catalog, orders]
```

1. Aggiungi il servizio **gateway**
2. In orders: **CATALOG_URL** e depends_on catalog
3. Togli **ports** da catalog e orders



Che valore ha CATALOG_URL? Ricorda: dentro un container, localhost è il container stesso.

3.3 · Avvia il negozio

```
docker compose up --build
```

Poi apri **http/shop.http** e invia le richieste dall'alto in basso.

```
gateway-1 | ... "GET /api/v1/products HTTP/1.1"
200 → 172.18.0.4:3001 0.004 s
gateway-1 | ... "POST /api/v1/orders HTTP/1.1"
201 → 172.18.0.5:3002 0.021 s
```

Richiesta	Atteso
POST /orders	201, confirmed
GET /products/p-1	scorte calate
99 monitor	409
p-999	422
.../reservations	403
:8080/qualcosa	404

3.4 · Un solo ingresso: verificalo

1

```
curl  
http://localhost:3001/products
```

Risponde? Perché?

2

Crea un prodotto da shop.http e
guarda l'header **Location**.

Che indirizzo contiene?

3

```
docker compose ps
```

**Quali servizi hanno una porta
nella colonna PORTS?**

4.1 · Due copie di catalog

```
docker compose up -d --scale catalog=2 \
  --no-recreate
docker compose ps
docker compose restart gateway
docker compose logs -f gateway
```

Ora invia più volte **GET /products** da shop.http.

1. Quanti indirizzi diversi compaiono dopo la freccia?
2. Perché serve riavviare il gateway?
3. Con la porta 3001 pubblicata, lo scale avrebbe funzionato?

4.2 · Spegni catalog

```
docker compose stop catalog
# ora da shop.http:
# POST /orders → ?
# GET /products → ?
docker compose logs orders
docker compose start catalog
```

Richiesta	Codice	Chi lo decide
POST /orders	503	orders
GET /products	502	il gateway

orders è vivo e risponde, ma senza catalog non può lavorare. Il gateway invece non trova proprio nessuno a cui passare la richiesta.

Checkpoint

Verifica	Esito atteso
npm test in services/orders	pass 13, fail 0
npm test in services/catalog	tutti verdi, 2 saltati senza DATABASE_URL
docker compose ps	quattro servizi, porte solo su gateway e catalog-db
POST /api/v1/orders	201, confirmed, scorte calate in catalog
POST /api/v1/products/p-1/reservations	403 dal gateway
curl localhost:3001/products	nessuna risposta

Consegna: l'output di npm test di orders e il file gateway/nginx.conf.

Se qualcosa non va

Sintomo	Causa e rimedio
Ogni ordine risponde 503	CATALOG_URL manca o usa localhost: <code>http://catalog:3001</code>
404 da catalog attraverso il gateway	<code>proxy_pass</code> senza <code>/products</code> : il prefisso non viene sostituito
gateway esce: <code>host not found in upstream</code>	Nome sbagliato in <code>nginx.conf</code> , o manca <code>depends_on</code>
<code>nginx: unexpected "}"</code>	Manca un punto e virgola nella riga prima
Modifico <code>nginx.conf</code> e non cambia nulla	<code>docker compose restart gateway</code>
502 dopo aver riavviato catalog	Il gateway ha il vecchio IP: <code>restart gateway</code>
Il POST arriva vuoto a catalog: 400	Manca <code>content-type: application/json</code> in <code>fetch</code>

ULTIMI 10 MINUTI

Cosa si rompe se...?

...catalog impiega 30 secondi a rispondere?

...orders si spegne dopo aver prenotato le scorte, ma prima di salvare l'ordine?

...si spegne il gateway?

PROSSIMA LEZIONE

Lezione 4 • Resilienza e configurazione

Rallentiamo catalog e guardiamo orders bloccarsi. Poi timeout, retry, circuit breaker, health check e log con correlation ID.

Per la volta prossima: `npm test` verde in orders e il gateway funzionante.

PER CHI FINISCE PRIMA, O PER CASA

Esercizi extra

#	Esercizio	Livello	Dove
1	Compensazione: rilasciare le scorte	medio	catalog, orders
2	Una chiamata invece di N	medio	catalog, orders
3	Limitare le richieste nel gateway	base	nginx.conf
4	Una versione 2 dell'API	avanzato	catalog, nginx
5	Progetta le API di notifications	carta e penna	—

Testi in [lezione-3/extra/README.md](#). Per 1, 2 e 4 ci sono test pronti; le soluzioni sono in [lezione-3/extra/soluzione](#).

Compensazione

Se la seconda riga non ha scorte, la prima resta prenotata. Sistemalo:

1. catalog: **POST /products/:id/releases** rimette le scorte
2. client: **release(id, quantity)**
3. POST /orders: se una prenotazione fallisce, rilascia le precedenti
4. gateway: blocca anche /releases

Domanda

E se catalog si spegne proprio tra la prenotazione e il rilascio?

Una chiamata invece di N

Un ordine di 10 righe fa 10 chiamate GET, una dopo l'altra.

1. catalog: **GET /products?ids=p-1,p-2**
2. client: **getProducts(ids)**, con URLSearchParams
3. POST /orders: una sola lettura, poi una Map per id

Domanda

Le prenotazioni restano una per riga. Come le ridurresti a una? Cosa diventerebbe più semplice?

Limitare le richieste nel gateway

Al massimo **5 richieste al secondo** per IP sugli ordini, poi **429 Too Many Requests**.

```
limit_req_zone $binary_remote_addr
    zone=ordini:1m rate=5r/s;
limit_req zone=ordini burst=5 nodelay;
```

```
for i in $(seq 1 15); do curl -s -o /dev/null \
    -w "%{http_code} " localhost:8080/api/v1/orders; done
```

Domanda

Perché conviene farlo nel gateway e non in ogni servizio?

Una versione 2 dell'API

Il sito vuole il prezzo in euro. Senza rompere la v1:

```
GET /api/v2/products/p-1
{ "price": { "amount": 89.9,
             "currency": "EUR" },
  "available": true, ... }
```

1. catalog: **GET /v2/products/:id**
2. gateway: **/api/v2/products**

Domande

Aggiungere **category** alla v1 richiede una v2? E togliere **description**?

Progetta le API di notifications

Nella lezione 7 arriva **notifications**, che manda le email di conferma.

1. Scrivi le sue rotte: metodo, percorso, corpo, codici
2. Quali dati di un ordine le servono? Li chiede o li riceve?
3. Se orders la chiamasse via HTTP, cosa succederebbe con notifications spento?

Tienilo da parte

Lo riprendiamo nella lezione 7, quando la notifica diventerà un evento.

Per approfondire

M. Fowler · *Microservices, sezione dati*

martinfowler.com: decentralized data management

C. Richardson · *Database per service*

microservices.io: il pattern e le alternative

C. Richardson · *API Gateway, BFF*

microservices.io: gateway e Backend for Frontend

M. Fowler · *Tolerant Reader*

martinfowler.com/bliki: consumatori robusti

Nginx Docs · *ngx_http_proxy_module*

nginx.org: proxy_pass, proxy_redirect

Docker Docs · *Networking in Compose*

Reti e nomi dei servizi in Compose
